

Centro Universitário de João Pessoa - Unipê  
Curso de Ciência da Computação  
Disciplina: Análise e Projeto de Sistemas I



# SaaS Barbearia

Documentação de Análise e Projeto de Sistemas

## **Autores**

Luis Henrique Âncoras Paiva  
Ryan Fernandes Ribeiro

Professor: Ricardo Roberto  
4º Período | 2026

## **Sumário**

---

### **1 – Introdução**

### **2 – Modelo de Negócio**

### **3 – Requisitos do Sistemas**

#### **3.1 – Requisitos Funcionais**

#### **3.2 – Requisitos Não-Funcionais**

### **4 – Diagramas UML**

#### **4.1 – Diagrama de Casos de Uso**

#### **4.2 – Diagrama de Classes**

#### **4.3 – Diagrama de Atividades**

#### **4.4 – Diagrama de Sequência**

### **5 – Arquitetura do Sistemas**

#### **5.1 – Visão Geral**

#### **5.2 – Segurança**

### **6 – Conclusão**

### **7 – Referências**

# 1. Introdução

---

Este documento apresenta a documentação completa de análise e projeto do sistema SaaS Barbearia, desenvolvido como trabalho prático da disciplina de Análise e Projeto de Sistemas I. O projeto tem como objetivo proporcionar uma solução tecnológica moderna para a gestão de agendamentos em barbearias, eliminando processos manuais e oferecendo uma experiência digital eficiente tanto para clientes quanto para barbeiros.

A documentação reúne todos os artefatos produzidos durante a fase de análise e projeto, abrangendo desde a descrição do modelo de negócio e levantamento de requisitos até os diagramas UML que representam o comportamento e a estrutura do sistema.

## 2. Modelo de Negócio

---

O sistema consiste num SaaS (Software as a Service) voltado ao agendamento de horários em barbearias. Seu objetivo principal é informatizar e automatizar a gestão de marcações, substituindo processos manuais (agenda física, WhatsApp, ligações) por uma plataforma web centralizada e de fácil acesso.

O sistema disponibiliza duas visões distintas e com permissões separadas:

- Visão do Cliente: interface voltada ao cliente final, permitindo que ele realize agendamentos de forma autônoma, escolhendo o barbeiro, o serviço desejado, a data e o horário disponíveis, além de consultar o histórico dos próprios agendamentos.
- Visão do Barbeiro (Admin): painel administrativo completo com controle total sobre a agenda, gestão de serviços, cadastro de clientes, gerenciamento de disponibilidade e cancelamento ou exclusão de agendamentos.

A plataforma é construída com as seguintes tecnologias:

- Backend: Java com Spring Boot (API REST + Spring Security para autenticação JWT)
- Frontend: React.js
- Banco de Dados: PostgreSQL

## 3. Requisitos do Sistema

---

### 3.1 Requisitos Funcionais

A tabela a seguir apresenta todos os requisitos funcionais levantados para o sistema:

| Código | Nome                                 | Ator               | Descrição                                                                                                                                                             |
|--------|--------------------------------------|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RF01   | Autenticar Usuário                   | Cliente / Barbeiro | O sistema deve fornecer mecanismo de autenticação (Login) com distinção estrita de permissões entre os perfis de Cliente e Barbeiro.                                  |
| RF02   | Gerenciar Cliente                    | Barbeiro / Cliente | Permitir o registro, edição e consulta de clientes, guardando dados como nome, telefone e e-mail.                                                                     |
| RF03   | Realizar Agendamento                 | Cliente / Barbeiro | Permitir a criação de um agendamento associando Cliente, Barbeiro, Serviço, Data e Hora. O cliente agenda para si mesmo; o barbeiro pode criar para qualquer cliente. |
| RF04   | Visualizar Agendamentos              | Cliente            | Permitir que o cliente visualize apenas os seus próprios agendamentos (histórico passado e horários futuros), sem a opção de cancelamento autônomo.                   |
| RF05   | Gerenciar Agendamentos               | Barbeiro           | Permitir exclusivamente ao barbeiro o cancelamento ou exclusão de agendamentos existentes.                                                                            |
| RF06   | Consultar Agenda Global              | Barbeiro           | Permitir que o barbeiro consulte a lista de todos os agendamentos marcados para uma determinada data, com visão completa de horários livres e ocupados.               |
| RF07   | Gerenciar Disponibilidade e Barbeiro | Barbeiro           | Permitir cadastrar, editar, inativar e consultar dados dos barbeiros, além de gerenciar as datas disponíveis e blocos de horários (início e fim).                     |
| RF08   | Gerenciar Tipo de Serviço            | Barbeiro           | Permitir a gestão dos serviços oferecidos, incluindo nome, preço e tempo de duração estimado.                                                                         |

### 3.2 Requisitos Não-Funcionais

| Código | Categoria               | Descrição                                                                                                                                                                                                   |
|--------|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| RNF01  | Segurança e Autorização | A senha de todos os usuários deve ser criptografada no banco de dados em 100% dos casos. A API do backend deve bloquear tentativas de clientes acessarem rotas exclusivas do barbeiro (HTTP 403 Forbidden). |
| RNF02  | Desempenho              | O tempo de resposta de todas as telas do sistema deve ser menor ou igual a 2 segundos em 90% dos casos.                                                                                                     |
| RNF03  | Legal / Privacidade     | O sistema deve garantir a privacidade dos dados pessoais dos clientes (nome, telefone e e-mail) em conformidade com a LGPD (Lei Geral de Proteção de Dados).                                                |

## 4. Diagramas UML

Esta seção apresenta os artefatos visuais produzidos na fase de análise e projeto do sistema, representados por meio da linguagem UML (Unified Modeling Language).

### 4.1 Diagrama de Casos de Uso

O diagrama de casos de uso define os atores do sistema e os casos de uso disponíveis para cada perfil, evidenciando as fronteiras do sistema e as interações possíveis entre os usuários e as funcionalidades disponibilizadas.

Os dois atores principais são:

- Cliente: pode se autenticar, gerenciar seus dados, realizar agendamentos e visualizar seus próprios agendamentos.
- Barbeiro (Admin): além de compartilhar os casos de uso de autenticação e agendamento, possui acesso exclusivo às funcionalidades de gerenciamento global — agenda, disponibilidade, serviços e cancelamento de agendamentos.

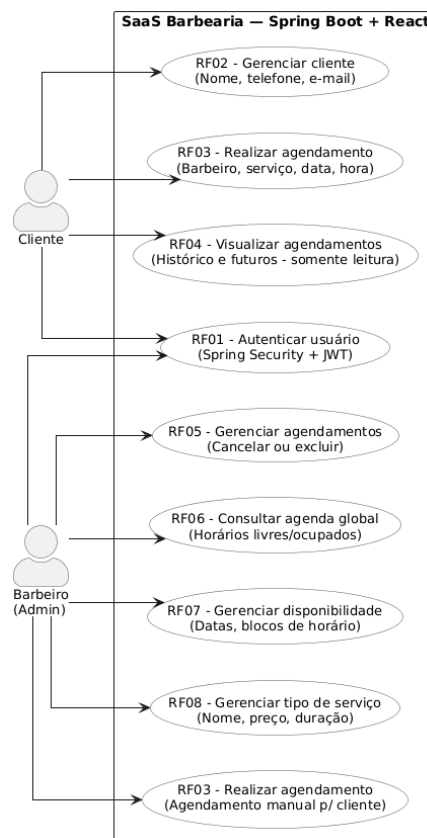


Figura 1 – Diagrama de Casos de Uso do Sistema SaaS Barbearia

## 4.2 Diagrama de Classes

O diagrama de classes modela a estrutura estática do sistema, apresentando as entidades do domínio, seus atributos, métodos e os relacionamentos entre elas.

As principais classes identificadas são:

- **Usuario** (abstrata): classe base com atributos comuns de autenticação (id, email, senhaCriptografada) e o método autenticar(). É especializada pelas classes Cliente e Barbeiro.
- **Cliente**: estende Usuario e adiciona nome e telefone. Possui métodos para cadastro, edição, consulta e visualização dos próprios agendamentos.
- **Barbeiro**: estende Usuario com nome e telefone. Além dos métodos de CRUD, possui o método consultarAgendaGlobal(data) que retorna todos os agendamentos de uma data.
- **Servico**: entidade independente com id, nome, preco e tempoDuracaoEstimado, gerenciada pelo barbeiro.
- **Agendamento**: entidade central que associa Cliente, Barbeiro e Servico a uma data, hora e status. Contém os métodos realizarAgendamento() e cancelarAgendamento().
- **Disponibilidade**: entidade associada ao Barbeiro que define os blocos de horários de trabalho (dataTrabalho, horarioInicio, horarioFim).

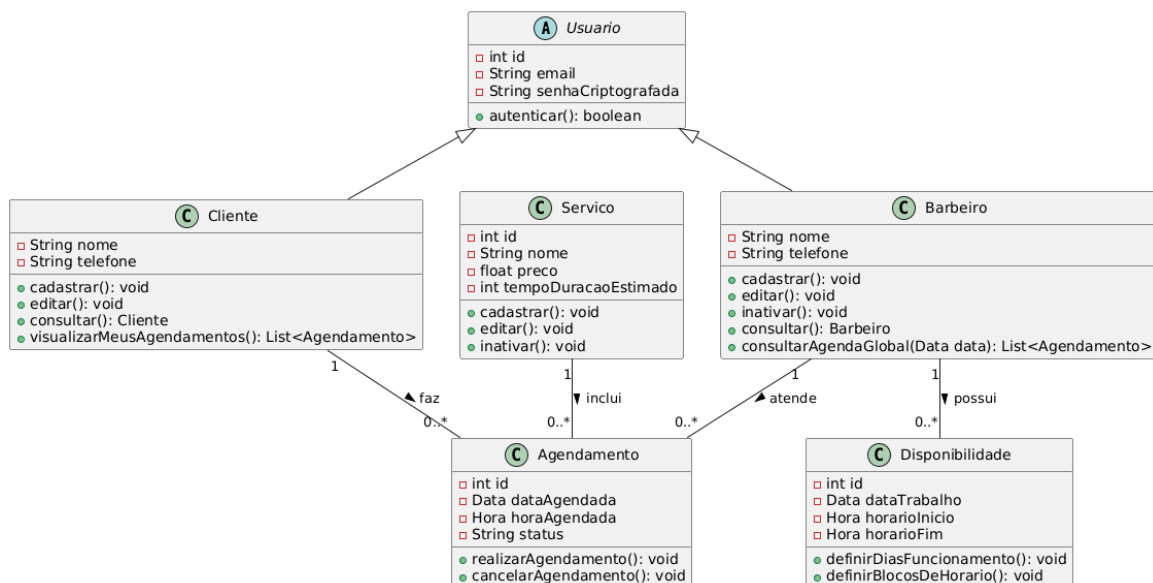


Figura 2 – Diagrama de Classes do Sistema SaaS Barbearia

### 4.3 Diagrama de Atividades

O diagrama de atividades representa o fluxo de controle do sistema após a autenticação do usuário, modelando as ações realizadas por cada perfil de forma paralela através de swimlanes.

O fluxo principal ocorre da seguinte forma:

- O usuário acessa o sistema pelo frontend React e preenche as credenciais de login.
- O Spring Security valida as credenciais (RF01). Caso inválidas, o fluxo retorna para a tela de login.
- Com credenciais válidas, o sistema verifica o perfil do usuário e redireciona: Clientes acessam o Dashboard do Cliente; Barbeiros acessam o Painel Administrativo.
- No Dashboard do Cliente: é possível visualizar o histórico de agendamentos (RF04) ou iniciar um novo agendamento selecionando barbeiro, serviço, data e hora (RF03).
- No Painel do Barbeiro: estão disponíveis as ações de consultar a agenda global (RF06), gerenciar serviços (RF08), gerenciar disponibilidade (RF07), cancelar/excluir agendamentos (RF05) e realizar agendamentos manuais (RF03).

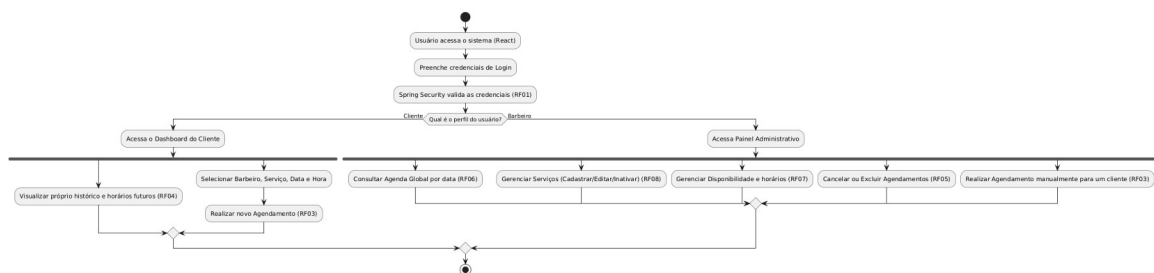


Figura 3 – Diagrama de Atividades do Sistema SaaS Barbearia

## 4.4 Diagrama de Sequência

O diagrama de sequência detalha a interação dinâmica entre os componentes do sistema durante a execução do caso de uso RF03 – Realizar Agendamento pelo Cliente. Ele demonstra a troca de mensagens entre os objetos ao longo do tempo.

O fluxo modelado é:

- O Cliente seleciona barbeiro, serviço, data e hora no Frontend (React) e clica em "Confirmar Agendamento".
- O Frontend envia uma requisição POST /api/agendamentos com o token JWT e os dados do agendamento para a API REST (Spring Boot).
- O filtro de segurança JWT (RF01/RNF01) valida o token e as permissões do usuário antes de qualquer processamento.
- A API consulta o PostgreSQL para verificar disponibilidade: `SELECT count(*) FROM agendamentos WHERE barbeiro_id = X AND data = Y AND hora = Z`.
- [Horário já ocupado]: O banco retorna contagem > 0 e a API responde com HTTP 409 Conflict, exibindo alerta de erro na interface.
- [Horário disponível]: O banco retorna 0 e a API executa o `INSERT INTO agendamentos (RF03)`, respondendo com HTTP 201 Created e redirecionando o cliente para "Meus Agendamentos".

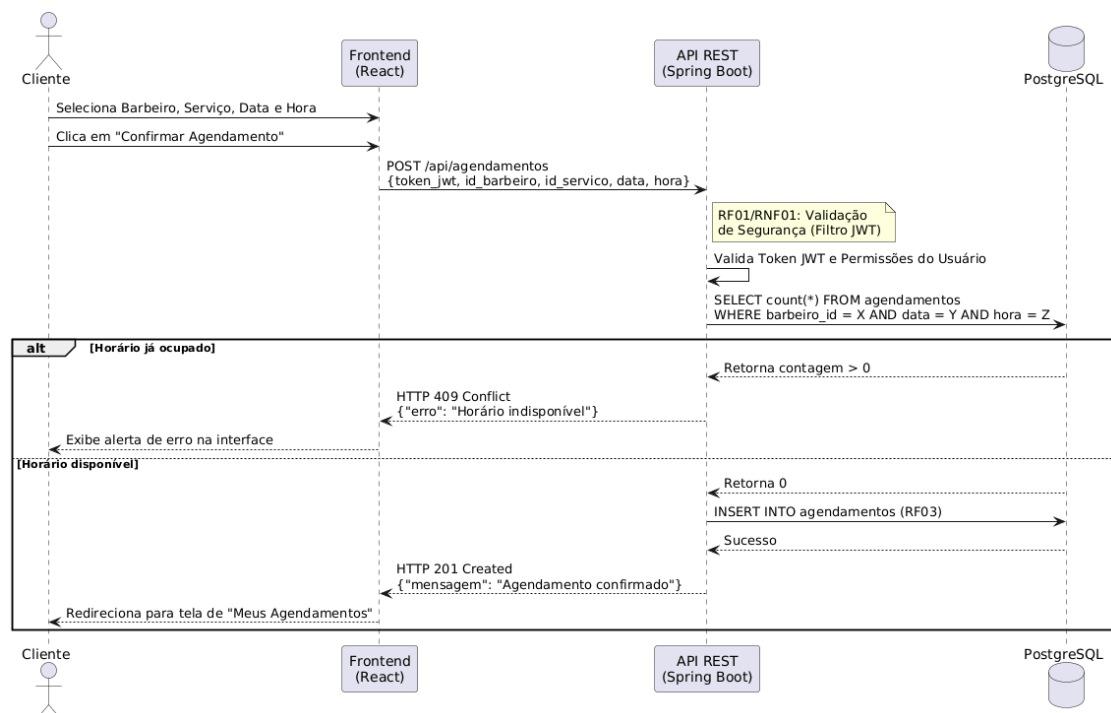


Figura 4 – Diagrama de Sequência: Realizar Agendamento (RF03)



## 5. Arquitetura do Sistema

### 5.1 Visão Geral

O sistema segue uma arquitetura de três camadas (3-tier architecture) com separação clara entre frontend, backend e banco de dados:

| Camada                  | Tecnologia         | Responsabilidade                                                                                     |
|-------------------------|--------------------|------------------------------------------------------------------------------------------------------|
| Apresentação (Frontend) | React.js           | Interface do usuário, roteamento, consumo da API REST, armazenamento do token JWT.                   |
| Aplicação (Backend)     | Java + Spring Boot | API REST, regras de negócio, autenticação JWT (Spring Security), controle de autorização por perfil. |
| Dados (Banco)           | PostgreSQL         | Persistência de todos os dados do sistema: usuários, agendamentos, serviços e disponibilidade.       |

### 5.2 Segurança

A segurança do sistema é implementada em múltiplas camadas, atendendo aos requisitos RNF01:

- Autenticação via JWT (JSON Web Token): ao realizar o login, o backend gera e retorna um token JWT assinado. O frontend armazena esse token e o inclui no cabeçalho de todas as requisições subsequentes.
- Autorização por perfil: o Spring Security intercepta todas as requisições e verifica o perfil do usuário extraído do JWT. Tentativas de clientes acessarem rotas restritas ao barbeiro resultam em HTTP 403 Forbidden.
- Criptografia de senha: todas as senhas são armazenadas no banco de dados utilizando o algoritmo BCrypt, garantindo que senhas em texto puro nunca sejam persistidas.
- Conformidade com LGPD (RNF03): dados pessoais dos clientes são tratados com respeito às diretrizes da Lei Geral de Proteção de Dados.

## 6. Conclusão

---

A documentação produzida neste trabalho evidencia uma análise completa e consistente do sistema SaaS Barbearia. Os artefatos gerados — diagrama de casos de uso, diagrama de classes, diagrama de atividades e diagrama de sequência — demonstram de forma clara a estrutura, o comportamento e as interações do sistema proposto.

O levantamento de requisitos funcionais (RF01 a RF08) e não-funcionais (RNF01 a RNF03) estabelece uma base sólida para a fase de implementação, garantindo que aspectos críticos como segurança, desempenho e privacidade sejam contemplados desde a concepção do projeto.

A escolha das tecnologias — Java/Spring Boot para o backend, React para o frontend e PostgreSQL como banco de dados relacional — está alinhada com as melhores práticas do mercado e com os requisitos de robustez e escalabilidade esperados de um produto SaaS.

## 7. Referências

---

- FOWLER, Martin. UML Distilled: A Brief Guide to the Standard Object Modeling Language. 3. ed. Addison-Wesley, 2003.
- PRESSMAN, Roger S. Engenharia de Software: Uma Abordagem Profissional. 8. ed. McGraw-Hill, 2016.
- Spring Boot Documentation. Disponível em: <https://spring.io/projects/spring-boot>
- React Documentation. Disponível em: <https://react.dev>
- PostgreSQL Documentation. Disponível em: <https://www.postgresql.org/docs/>
- LGPD – Lei Geral de Proteção de Dados (Lei nº 13.709/2018). Disponível em: <https://www.planalto.gov.br>